

Vertex Cover Approximation Exercises

Question-Answer Notes for Exercises 35.1-1 to 35.1-5

Background

Algorithm Reminder

APPROX-VERTEX-COVER repeatedly chooses an uncovered edge (u, v) , puts both endpoints u and v into the cover, and then removes all edges incident on either u or v .

The key proof idea behind the 2-approximation is that the edges chosen by the algorithm form a matching. Since every vertex cover must include at least one endpoint from each edge in that matching, the matching gives a lower bound on the optimum.

1 Exercise 35.1-1

Question 35.1-1

Give an example of a graph for which APPROX-VERTEX-COVER always yields a suboptimal solution.

Answer

Use the graph with two vertices and one edge:

$$a - b.$$


The optimal vertex cover has size 1, for example

$$C^* = \{a\} \quad \text{or} \quad C^* = \{b\}.$$

APPROX-VERTEX-COVER has only one possible edge to choose, namely (a, b) . It therefore adds both endpoints and returns

$$C = \{a, b\}.$$

Thus

$$|C| = 2 \quad \text{and} \quad \text{OPT} = 1.$$

The algorithm is always suboptimal on this graph.

General Family

More generally, take k disjoint edges on $2k$ vertices. The optimum chooses one endpoint from each edge, so $\text{OPT} = k$. APPROX-VERTEX-COVER chooses both endpoints of each edge, so it returns $2k$ vertices. Hence the approximation is exactly a factor of 2 away from optimum.

2 Exercise 35.1-2

Question 35.1-2

Prove that the set of edges picked in APPROX-VERTEX-COVER forms a maximal matching in the graph G .

Answer

Let A be the set of edges selected by APPROX-VERTEX-COVER.

Proof

First, A is a matching. When the algorithm chooses an edge (u, v) , it removes every edge incident on u or v . Therefore, no later chosen edge can share an endpoint with (u, v) . Hence no two edges in A share an endpoint.

Second, A is maximal. When the algorithm terminates, there are no uncovered edges left. Every edge not chosen by the algorithm was removed because it was incident on some chosen edge. Therefore every edge of G either belongs to A or shares an endpoint with an edge in A .

So no additional edge can be added to A without breaking the matching property. Thus A is a maximal matching.

Maximal vs. Maximum

Maximal means no more edges can be added to the matching. **Maximum** means the matching has the largest possible number of edges. APPROX-VERTEX-COVER produces a maximal matching, not necessarily a maximum matching.

3 Exercise 35.1-3

Question 35.1-3

Professor Bündchen proposes the following heuristic for the vertex-cover problem: repeatedly select a vertex of highest degree, and remove all of its incident edges. Give an example showing that this heuristic does not have an approximation ratio of 2.

Answer

The highest-degree heuristic can be forced to choose the wrong side of a bipartite graph. We construct a graph with bipartition

$$V = R \cup L$$

such that R itself is a vertex cover, but the heuristic can select all vertices in L , with $|L| > 2|R|$.

Construction

Fix

$$R = \{r_1, r_2, \dots, r_n\},$$

where n is large. Start with $L = \emptyset$. For each $i = 2, 3, \dots, n$, do the following:

1. Let

$$k_i = \left\lfloor \frac{n}{i} \right\rfloor.$$

2. Choose a set $S_i \subseteq R$ of size $k_i i$ among the currently lowest-degree vertices of R .

3. Partition S_i into k_i groups of size i .

4. For each group, add one new vertex to L and connect it to exactly the i vertices of that group.

Thus, for each i , we add $k_i = \lfloor n/i \rfloor$ vertices of degree i to L .

Because each step increases the degrees of currently lowest-degree vertices in R , the degrees of vertices in R always differ by at most 1. At the end, add degree-1 vertices to L adjacent to the lower-degree vertices of R until all vertices in R have the same degree.

Important Structural Property

For each degree i , a vertex of R is adjacent to at most one degree- i vertex of L . This is because, during the degree- i construction step, each selected vertex of R appears in exactly one group.

Why There Is a Small Vertex Cover

The graph is bipartite and every edge has one endpoint in R and one endpoint in L . Therefore R is a vertex cover. Hence

$$\text{OPT} \leq |R| = n.$$

Why the Heuristic Can Choose All of L

Consider the heuristic with ties broken in favor of vertices in L . The highest degree among vertices in L is initially n , because the construction adds one degree- n vertex.

After all vertices of degree greater than i in L have been selected and their incident edges removed, any remaining vertex in R has at most one remaining neighbor of each degree $1, 2, \dots, i$. Therefore its remaining degree is at most i .

At the same time, there is still a remaining vertex in L of degree i . Thus a highest-degree choice can always select a vertex from L . Repeating this argument from degree n down to degree 1, the heuristic can select every vertex in L .

Why This Beats Factor 2

Ignoring the final degree-1 vertices, the number of vertices added to L is already

$$|L| \geq \sum_{i=2}^n \left\lfloor \frac{n}{i} \right\rfloor.$$

Using $\lfloor x \rfloor \geq x - 1$,

$$\sum_{i=2}^n \left\lfloor \frac{n}{i} \right\rfloor \geq n \sum_{i=2}^n \frac{1}{i} - (n-1).$$

Since $\sum_{i=1}^n 1/i \geq \ln(n+1)$,

$$|L| \geq n(\ln(n+1) - 2) + 1.$$

For sufficiently large n , this is greater than $2n$. For example, $n = 100$ already gives $|L| > 2n$.

Since $\text{OPT} \leq n$, the heuristic can return more than 2OPT vertices.

Conclusion

The highest-degree heuristic does not have approximation ratio 2. In fact, because the ratio grows on the order of $\ln n$, this heuristic is not a ρ -approximation for any fixed constant ρ under such legal tie choices.

4 Exercise 35.1-4**Question 35.1-4**

Give an efficient greedy algorithm that finds an optimal vertex cover for a tree in linear time.

Answer

If the tree has one vertex, return the empty cover. Otherwise, repeatedly choose the neighbor of a leaf.

Greedy-Vertex-Cover for a Tree

1. Initialize $C \leftarrow \emptyset$.
2. Put all current leaves into a queue Q .
3. While the remaining tree has at least one edge:
 - a. Pop a current leaf v from Q .
 - b. Let u be the only current neighbor of v .
 - c. Add u to C .
 - d. Delete u , v , and all edges incident on u from the remaining forest.
 - e. Whenever a vertex becomes a leaf, insert it into Q .
4. Return C .

Why This Greedy Choice Is Safe

Let v be a leaf and let u be its only neighbor. The edge (u, v) must be covered, so every vertex cover contains at least one of u and v .

We claim that there is always an optimal vertex cover that does not contain the leaf v . Let C^* be an optimal vertex cover.

If $v \notin C^*$, the claim is already true. If $v \in C^*$, then $u \notin C^*$; otherwise we could remove v and still cover every edge, contradicting optimality. Now replace v by u :

$$C' = (C^* - \{v\}) \cup \{u\}.$$

This replacement does not uncover any edge:

- the leaf v only touches the edge (u, v) ;
- u covers (u, v) and all other edges incident on u .

Thus C' is also an optimal vertex cover, and it contains u instead of v .

Therefore choosing the neighbor of a leaf is a safe greedy choice.

Correctness and Running Time

After choosing u , all edges incident on u are covered, so those edges can be deleted. The remaining graph is a forest. Applying the same leaf argument inductively proves that the algorithm returns an optimal vertex cover.

For linear time, store the tree with adjacency lists and maintain current degrees. Initially finding all leaves takes $\mathcal{O}(|V|)$ time. Each edge is deleted once, and each degree update takes constant time. Therefore the total running time is

$$\mathcal{O}(|V| + |E|).$$

For a tree, $|E| = |V| - 1$, so the running time is linear:

$$\mathcal{O}(|V|).$$

5 Exercise 35.1-5**Question 35.1-5**

From the proof of Theorem 34.12, we know that the vertex-cover problem and the **NP**-complete clique problem are complementary: an optimal vertex cover is the complement of a maximum-size clique in the complement graph. Does this imply that there is a polynomial-time approximation algorithm with a constant approximation ratio for the clique problem? Justify your answer.

Answer

No. The complement relationship preserves exact optimal solutions, but it does not preserve approximation ratios.

Let G be a graph with n vertices, and let $\overline{G}$ be its complement. A set C is a vertex cover in G if and only if $V - C$ is a clique in $\overline{G}$. Therefore:

$$\omega(\overline{G}) = n - \tau(G),$$

where $\omega(\overline{G})$ is the maximum clique size in $\overline{G}$ and $\tau(G)$ is the minimum vertex-cover size in G .

This identity is exact, but approximation is multiplicative. Suppose the minimum vertex cover in G has size

$$\tau(G) = k.$$

A 2-approximation for vertex cover might return a cover of size $2k$. The corresponding clique in $\overline{G}$ would then have size

$$n - 2k,$$

while the true maximum clique size is

$$n - k.$$

For this complement transformation to give a constant-factor approximation for clique, there would need to be some constant $\lambda > 0$ such that

$$n - 2k \geq \lambda(n - k)$$

for all relevant values of n and k .

Now choose

$$k = \frac{n}{2} - \epsilon.$$

Then the clique produced from the approximate vertex cover has size

$$n - 2k = 2\varepsilon,$$

whereas the maximum clique has size

$$n - k = \frac{n}{2} + \varepsilon.$$

The ratio is

$$\frac{2\varepsilon}{n/2 + \varepsilon}.$$

For fixed small ε , this ratio goes to 0 as n grows. Therefore no positive constant λ is guaranteed.

Why the Ratio Breaks

A multiplicative approximation to $\tau(G)$ does not become a multiplicative approximation to $n - \tau(G)$. Subtracting from n can destroy the approximation ratio completely.

Conclusion

The exact relationship

$$\text{minimum vertex cover in } G \iff \text{maximum clique in } \overline{G}$$

does not imply a constant-factor approximation algorithm for CLIQUE. The reason is that taking complements changes the objective value from τ to $n - \tau$, and multiplicative approximation guarantees do not survive this subtraction.

Final Summary

What to Remember

- A three-vertex path makes APPROX-VERTEX-COVER always suboptimal.
- The edges chosen by APPROX-VERTEX-COVER form a maximal matching.
- Choosing highest-degree vertices is not guaranteed to be a 2-approximation.
- Trees have an optimal linear-time greedy vertex-cover algorithm: repeatedly choose the neighbor of a leaf.
- Exact complement relationships do not automatically preserve approximation ratios.